

- Lab 07 – Visibility & Hidden Surface Removal with OpenGL

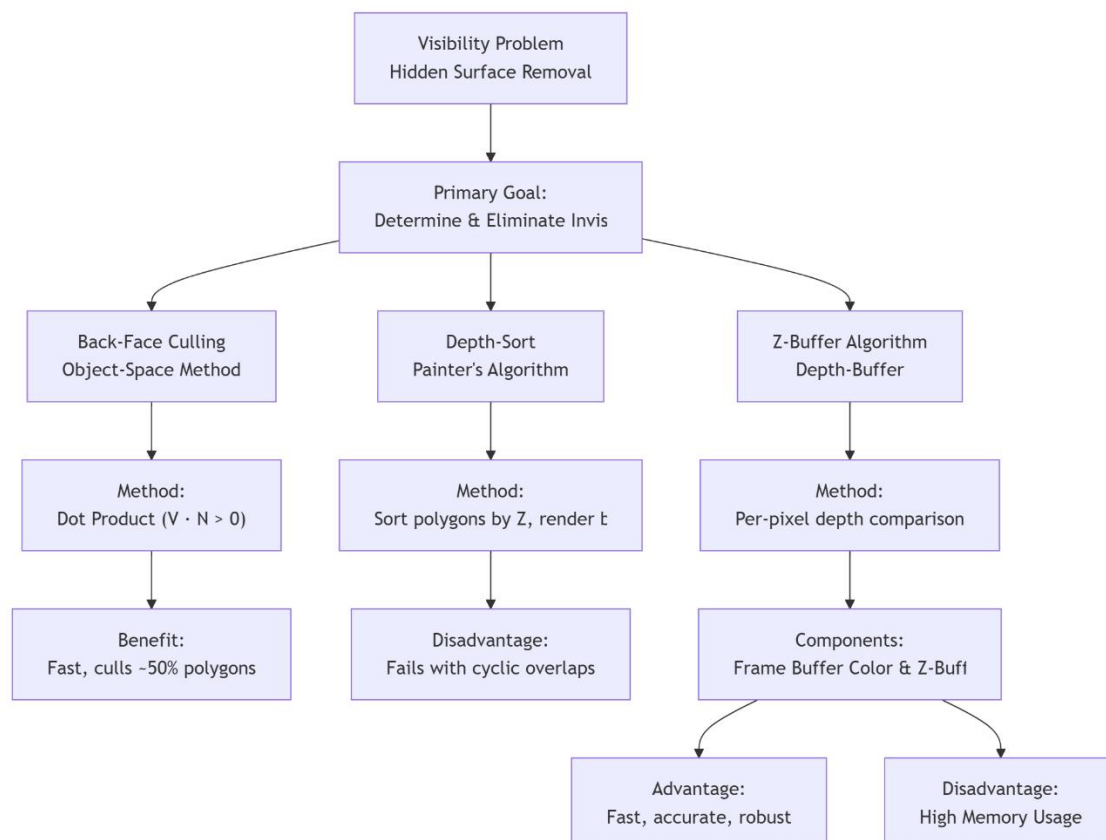
Objective:

By the end of this lab, students will:

- Understand and implement visibility determination techniques.
- Use OpenGL to visualize back-face culling and Z-buffering.
- Explain the concepts and code to earn marks.

The Visibility Problem

The **Visibility Problem**, or **Hidden Surface Removal**, is the challenge of determining which surfaces in a 3D scene are visible from a given viewpoint and which are hidden behind others. Instead of directly finding what is visible, the problem is efficiently solved by **eliminating invisible geometry**.



Back-Face Culling

This is an object-space method that removes polygons facing away from the viewer before they are processed further. It works by checking the angle between a polygon's normal vector and the view direction. If the polygon is facing away (the angle is less than 90 degrees), it is "culled" or discarded. This is highly efficient for closed, convex objects.

Additional note :

Where the "Magic Formula" Comes From

The formula is derived from calculating the **determinant of a special 3x3 matrix**. Let's build this matrix together.

Step 1: Set up the matrix

We write a 3x3 matrix where:

- **Row 1:** The unit vectors **i, j, k**
- **Row 2:** The components of vector **a** = $\langle a_1, a_2, a_3 \rangle$
- **Row 3:** The components of vector **b** = $\langle b_1, b_2, b_3 \rangle$

$$\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

Step 2: Expand along the first row

To find this determinant, we "expand" along the top row (i, j, k). The rule is:

$$\text{Determinant} = i \times (\text{det of minor}_{11}) - j \times (\text{det of minor}_{12}) + k \times (\text{det of minor}_{13})$$

Where the "minor" is the 2x2 matrix left when you remove the row and column of that element.

Step 3: Calculate each minor

For i (first element):

- Remove row 1 and column 1
- Minor = $\begin{vmatrix} a_2 & a_3 \\ b_2 & b_3 \end{vmatrix}$
- Determinant = $a_2b_3 - a_3b_2$
- So this gives us: $i(a_2b_3 - a_3b_2)$

For j (second element):

- Remove row 1 and column 2
- Minor = $\begin{vmatrix} a_1 & a_3 \\ b_1 & b_3 \end{vmatrix}$
- Determinant = $a_1b_3 - a_3b_1$
- Important: According to determinant rules, j gets a NEGATIVE sign
- So this gives us: $-j(a_1b_3 - a_3b_1)$

For k (third element):

- Remove row 1 and column 3
- Minor = $\begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix}$
- Determinant = $a_1b_2 - a_2b_1$
- So this gives us: $k(a_1b_2 - a_2b_1)$

Step 4: Put it all together

$$\mathbf{a} \times \mathbf{b} = i(a_2b_3 - a_3b_2) - j(a_1b_3 - a_3b_1) + k(a_1b_2 - a_2b_1)$$

If we write this in component form:

$$\mathbf{a} \times \mathbf{b} = \langle (a_2b_3 - a_3b_2), -(a_1b_3 - a_3b_1), (a_1b_2 - a_2b_1) \rangle$$

Which simplifies to our "magic formula":

$$\mathbf{a} \times \mathbf{b} = \langle (a_2b_3 - a_3b_2), (a_3b_1 - a_1b_3), (a_1b_2 - a_2b_1) \rangle$$

Quick Memory Aid

Here's an easy way to remember it without the matrix:

- **x-component:** $(a_2b_3 - a_3b_2) \rightarrow$ "y times z minus z times y"
- **y-component:** $(a_3b_1 - a_1b_3) \rightarrow$ "z times x minus x times z"
- **z-component:** $(a_1b_2 - a_2b_1) \rightarrow$ "x times y minus y times x"

Mathematical Foundation

A polygon is **back-facing** if:

$$\mathbf{V}_{\text{view}} \cdot \mathbf{N} > 0$$

Where:

- $\mathbf{N}$ is the polygon's normal vector.
- $\mathbf{V}_{\text{view}}$ is the vector from the camera to the polygon.

Example

1. Triangle Vertices

A(1, 1, 5), B(3, 1, 5), C(1, 3, 5)

2. Edge Vectors

$\mathbf{AB} = \mathbf{B} - \mathbf{A} = (3-1, 1-1, 5-5) = (2, 0, 0) \checkmark$

$\mathbf{AC} = \mathbf{C} - \mathbf{A} = (1-1, 3-1, 5-5) = (0, 2, 0) \checkmark$

3. Surface Normal (N)

Using cross product: $\mathbf{AB} \times \mathbf{AC}$

$$\mathbf{N} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 2 & 0 & 0 \\ 0 & 2 & 0 \end{vmatrix} = (0-0)\mathbf{i} - (0-0)\mathbf{j} + (4-0)\mathbf{k} = (0, 0, 4)$$

4. Centroid (G)

$$\mathbf{G} = \left(\frac{1+3+1}{3}, \frac{1+1+3}{3}, \frac{5+5+5}{3} \right) = \left(\frac{5}{3}, \frac{5}{3}, 5 \right)$$

5. View Direction (V)

From surface to camera: $\mathbf{E} - \mathbf{G}$

$$\mathbf{V} = (0, 0, 0) - \left(\frac{5}{3}, \frac{5}{3}, 5 \right) = \left(-\frac{5}{3}, -\frac{5}{3}, -5 \right)$$

6. Dot Product Test

$$\mathbf{N} \cdot \mathbf{V} = (0, 0, 4) \cdot \left(-\frac{5}{3}, -\frac{5}{3}, -5 \right) = 0 + 0 + (4 \times -5) = -20$$

7. Result Interpretation

$\mathbf{N} \cdot \mathbf{V} = -20 < 0 \checkmark$

Since the dot product is negative, the triangle is **back-facing** and should be culled.

Back-Face Culling in OpenGL

Task:

Create a simple 3D scene (e.g., cube or pyramid) and implement back-face culling.

Steps:

1. Initialize OpenGL window.
2. Enable back-face culling:

```
float angle = 0.0f;  
int showBackface = 1; // 1 = show backface, 0 = cull backface
```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glLoadIdentity();
    gluLookAt(0, 0, 3, 0, 0, 0, 1, 0);
    glRotatef(angle, 0, 1, 0);

    if (showBackface) {
        glDisable(GL_CULL_FACE);
    } else {
        glEnable(GL_CULL_FACE);
        glCullFace(GL_BACK);
    }

    // Draw a single quad that we can see from both sides
    glBegin(GL_QUADS);
    // Front side (red)

    // Back side (blue) - same vertices but different color

    glutSwapBuffers();
}

void reshape(int w, int h) {
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double)w / (double)h, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void keyboard(unsigned char key, int x, int y) {
    if (key == ' ') {
        showBackface = !showBackface;
        printf("Back-face culling: %s\n", showBackface ? "DISABLED" : "ENABLED");
    }
    glutPostRedisplay();
}

void idle() {
    angle += 1.0f;
    if (angle > 360) angle -= 360;
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Back-Face Culling Demo - Press SPACE to toggle");
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutKeyboardFunc(keyboard);
    glutIdleFunc(idle);

    glEnable(GL_DEPTH_TEST);

    printf("Press SPACE to toggle back-face culling\n");
    printf("Initial state: Back-face culling DISABLED (you see both sides)\n");

    glutMainLoop();
    return 0;
}

```

Show more lines

3. Render a 3D object and observe which faces are culled.

Depth Sort (Painter's) Algorithm

This is an image-space method that sorts all polygons based on their distance from the viewer (their Z-coordinate). Polygons are then rendered from farthest to closest, like a painter painting a background first. Closer polygons naturally overwrite farther ones in the frame buffer.

Mathematical Foundation

The core operation is sorting. For each polygon, a representative Z-value (like the closest vertex or the average Z) is calculated. The list of polygons is then sorted by this Z-value in descending order.

Worked Example

Let's sort three objects for rendering.

Given:

- Object A (a mountain): Farthest, with an average Z-value of -100
- Object B (a tree): Middle, with an average Z-value of -50
- Object C (a character): Closest, with an average Z-value of -10
-

Step 1: Gather Z-values.

Unsorted List: [A: Z=-100, B: Z=-50, C: Z=-10]

Step 2: Sort by Z-value in descending order (farthest first).

In descending order, -100 is smaller than -10, so we must remember that "farther" means more negative in a Right-Hand System where the camera looks down the -Z axis.

- Farthest: A (Z = -100)
- Middle: B (Z = -50)
- Closest: C (Z = -10)

Sorted Render Order: [A, B, C]

Step 3: Render in sorted order.

1. First, render Object A (the mountain). The frame buffer is filled with the mountain.
2. Then, render Object B (the tree). Pixels belonging to the tree overwrite the mountain pixels.
3. Finally, render Object C (the character). Pixels belonging to the character overwrite both the tree and mountain pixels.

The result is a correct image where the character is in front of the tree, which is in front of the mountain.

Limitation: This fails with cyclic overlaps (e.g., three polygons where each one is both in front of and behind another) or intersecting polygons, requiring complex splitting to resolve.

Task:

Simulate the Painter's Algorithm manually (no OpenGL depth buffer).

Steps:

1. Create multiple overlapping polygons.
2. Sort them by depth (Z-value).
3. Render them in sorted order using `glBegin(GL_POLYGON)`.

```
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // DISABLE depth testing to simulate Painter's Algorithm

    // Draw BLUE circle first (background)

    // Draw RED circle second (foreground) - overlaps blue

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutCreateWindow("Painter's Algorithm Demo");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Z-buffer Algorithm in OpenGL

This is the most common hidden surface removal algorithm. It works at the pixel level. In addition to the frame buffer that stores color, it uses a Z-buffer (or depth buffer) that stores the current depth (Z-value) for each pixel. When a new polygon is rendered, the Z-value of its current pixel is compared to the value stored in the Z-buffer. The pixel is only updated if the new polygon is closer.

Worked Example

Let's see how the Z-buffer resolves visibility for two triangles competing for the same pixel.

Given:

- A green triangle with a Z-value of -0.7 at pixel (100, 200).
- A red triangle with a Z-value of -0.3 at the same pixel (100, 200).
- The Z-buffer is initialized to 1.0 (the far plane).

Step-by-Step Pixel Processing:

Step	Action at Pixel (100, 200)	Frame Buffer Color	Z-buffer Value	Explanation
1	Initialization	Black (Background)	1.0	Buffers are cleared
2	Process Green Triangle	Green	-0.7	$-0.7 < 1.0$? Yes
3	Process Red Triangle	Green (unchanged)	-0.7 (unchanged)	$-0.3 < -0.7$? No

Result: Even if the red triangle is rendered *after* the green one, the correct color (green) remains in the frame buffer because the Z-buffer comparison prevented the farther red triangle from overwriting it. This automatically handles any complexity of overlapping shapes.

Task:

Use OpenGL's depth buffer to implement Z-buffering.

```
glEnable(GL_DEPTH_TEST);
glDepthFunc(GL_LESS);

#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Enable depth testing (Z-buffer)

    // Draw a red square FAR away

    // Draw a green square CLOSE

    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Z-Buffer Demo");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Evaluation & Submission

Instructions:

- Submit your code and screenshots.
- Each student will explain:
 - What each algorithm does
 - How their code implements it
 - Strengths and weaknesses of each method